

BEST-Rec v4: SBERT-Augmented EASE with Learned Content-to-CF Mapping for Cold-Item Recommendation

Sections 3 to 8 (Methodology, Datasets and Setup, Experiments, Discussion, Limitations, Conclusion)

3. Methodology

This section describes the BEST-Rec v4 architecture, which is a substantial departure from the transformer-based BEST-Rec v1-v3 lineage. Where the prior versions sought to extract rich predictive signal from a deep neural network with cross-attention fusion, v4 deliberately minimizes parameterization in favor of closed-form linear algebra, arguing that deep recommenders are systematically over-parameterized for the small-to-medium Amazon Reviews 2023 subsets considered. The architecture instead derives its predictive power from three sources: (i) the regularised closed-form item-item similarity of EASE (Steck, 2019), (ii) a content prior obtained from a frozen pre-trained sentence encoder, and (iii) a novel learned linear map that projects content embeddings into the collaborative behavior space of EASE for cold items. The resulting system has zero learned embeddings, four ridge-regression coefficients, and trains in seconds on the largest dataset considered.

Section 3.1 formalizes the prediction problem and notation. Section 3.2 walks through the data flow with reference to the architecture diagram (Figure 3.1). Sections 3.3 - 3.4 derive the closed-form solutions for the warm and cold paths. Section 3.5 describes the rating-prediction head used for MAE/RMSE evaluation. Section 3.6 describes the cold-user few-shot protocol. Sections 3.7 - 3.8 conclude with the training procedure and a complexity analysis.

3.1 Problem Formulation and Notation

Let $U = \{u_1, \dots, u_m\}$ be a set of m users and $I = \{i_1, \dots, i_n\}$ a set of n items. We observe a sparse set of interactions $\Omega \subset U \times I$, where $(u, i) \in \Omega$ iff user u has rated item i . Each observation is associated with an explicit rating $r_{ui} \in \{1, 2, 3, 4, 5\}$ (integer star rating) and an optional review-text string. Let $X \in \{0, 1\}^{m \times n}$ be the binary indicator matrix of Ω , with $X[u, i] = 1$ iff $(u, i) \in \Omega$.

Each item i has a textual title t_i obtained from the Amazon catalog metadata. We encode every title once via a frozen pre-trained sentence encoder $\phi : T \rightarrow \mathbb{R}^{d_s}$, in our experiments the all-MiniLM-L6-v2 Sentence-BERT model (Reimers and Gurevych, 2019) with output dimensionality $d_s = 384$. The resulting item embeddings $e_i = \phi(t_i)$ are used in two places: as a Bayesian-style content prior in the warm EASE Gram (Section 3.3) and as the input feature for the LC2C cold-item regression (Section 3.4). It is critical that ϕ is frozen and computed once before any train/test split because (i) e_i is derived from external metadata (the Amazon product catalog), not from interactions, so it cannot leak the train/test split, and (ii) recomputing it per fold would be wasteful and would produce identical results.

We evaluate three distinct prediction tasks, each with its own train/test split protocol described in Section 4.2:

- 1. Warm leave-one-out ranking.** For each user u with at least two interactions, hold out one interaction (u, i^*) per fold; train on $\Omega \setminus \{(u, i^*)\}$; for each held-out user, rank the held-out item i^* against all items the user has not seen and compute NDCG@10, HR@10, MRR.

2. Cold-user few-shot ranking. Held-out users contribute zero interactions to training. At test time, the user supplies a one-shot “context” signal — a single interaction whose item title is SBERT-encoded and used as the user’s content vector c_u . Rank u ’s remaining test items against all unseen items, with the context item excluded from prediction targets.

3. Cold-item GroupKFold ranking. Items are partitioned into 5 disjoint folds; in each fold, 20% of items are entirely removed from training. Held-out cold items have no training interactions whatsoever. For each test interaction (u, j) where j is cold, rank j against all other cold items.

We additionally report MAE and RMSE on rating prediction. The ranking score from the EASE-based model is real-valued and unbounded; to produce a calibrated rating $\hat{r}_{ui}$ in $[1, 5]$ we fit a small ridge regression on training pairs (Section 3.5). Table 3.1 summarizes all symbols used in the remainder of this paper.

Table 3.1: Notation summary.

Symbol	Definition
m, n	number of users / number of items
$X \in \{0,1\}^{m \times n}$	binary user-item interaction matrix
$\Omega \subseteq U \times I$	set of observed (user, item) pairs
r_{ui}	observed rating of u on i , integer in $[1, 5]$
t_i	item i title string
ϕ	frozen pre-trained sentence encoder (SBERT)
$e_i = \phi(t_i)$	item content embedding ($\mathbb{R}^{384}$)
S_{content}	item-item content cosine similarity matrix ($\mathbb{R}^{n \times n}$)
G	augmented Gram matrix $X^T X + \lambda I + \beta S_{\text{content}}$
B	EASE item-item similarity matrix ($\mathbb{R}^{n \times n}$)
λ	L2 regularization on the Gram matrix
β	weight of the SBERT content prior
μ	ridge regularization for the LC2C mapping
$W \in \mathbb{R}^{d_s \times n}$	LC2C learned content-to-CF map
$\hat{B}_{:, j_{\text{cold}}}$	predicted EASE column for cold item j
b_u, b_i	user / item rating bias (mean deviation)
gm	global mean training rating
$\hat{r}_{ui}$	predicted rating, clipped to $[1, 5]$

3.2 Architecture Overview

Figure 3.1 is the end-to-end architecture diagram for BEST-Rec v4. The system divides into six stages, with all blocks executable in either NumPy/SciPy (warm path, LC2C ridge regression, rating head) or in HuggingFace's Sentence-Transformers library (SBERT encoding). Stage 1 ingests the raw Amazon Reviews 2023 jsonl files. Stage 2 deduplicates by (user_id, parent_asin) keeping the latest rating, then applies per-dataset k-core filtering. Stage 3 produces three outputs in parallel: the binary interaction matrix X , the train/test splits for the three evaluation protocols, and the SBERT title embeddings. Stage 4 builds the augmented Gram matrix $G = X^T X + \lambda I + \beta S_{\text{content}}$ and solves it for the EASE similarity matrix B via Cholesky factorization. Stage 5 produces ranking scores along two branches: the warm branch directly computes $(X B)$, while the cold branch trains an LC2C ridge regression to project SBERT embeddings of unseen cold items into the collaborative behavior space, then computes scores for the cold items. Stage 6 produces the final reported metrics — NDCG@10, HR@10, and MRR for ranking, and MAE, RMSE for rating prediction.

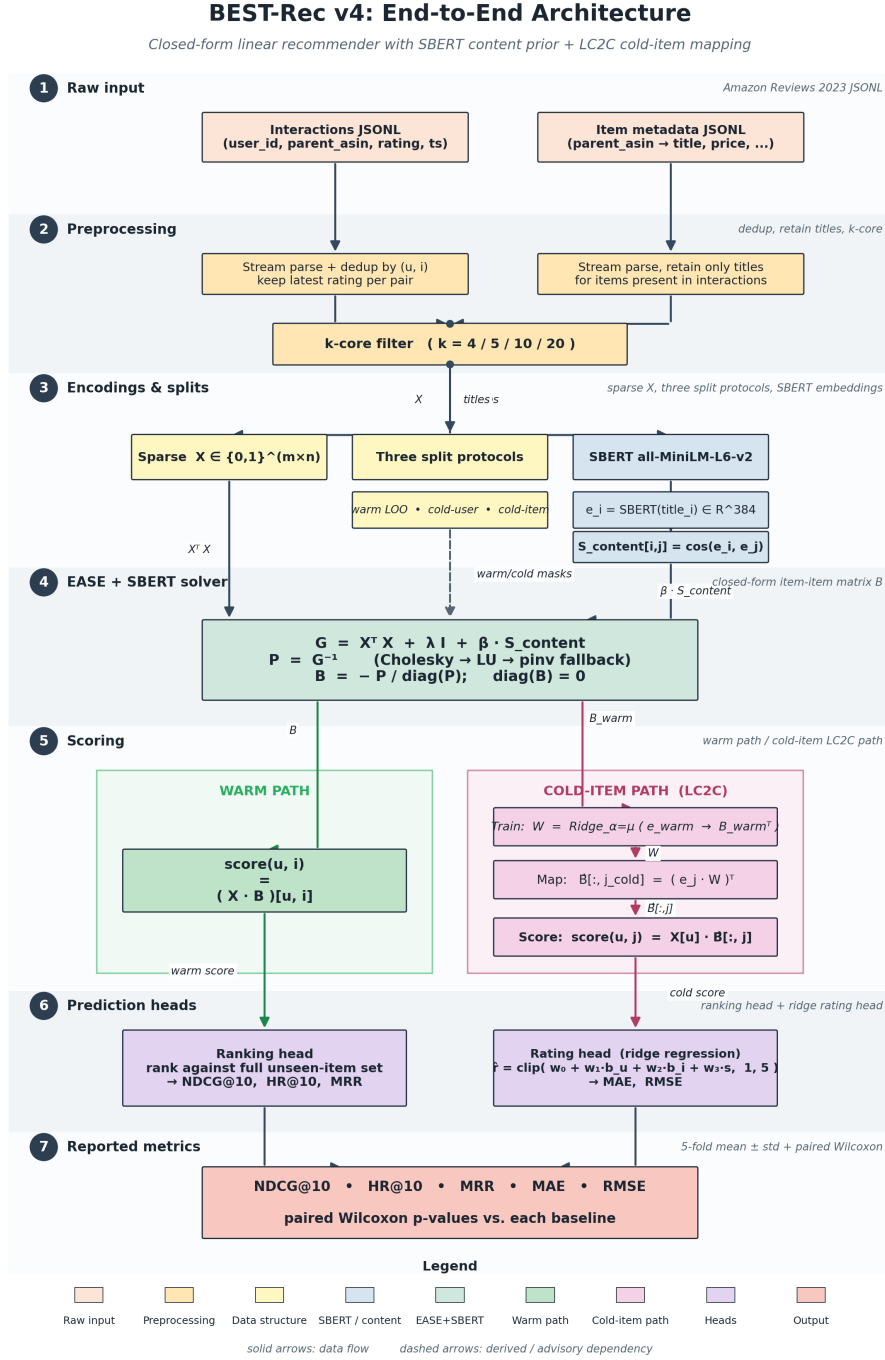


Figure 3.1: End-to-end architecture of BEST-Rec v4. Data flows top-to-bottom through six stages: ingest, preprocessing, encoding, EASE+SBERT, scoring, and prediction heads. The warm path (left, green) computes $X B$ for known users on warm items; the cold path (right, pink) uses ridge regression to predict B 's column for cold items from their SBERT title embeddings. Both paths feed into the ranking and rating heads. Hyperparameters: λ in $[30, 200]$, $\beta = 10$, $\mu = 1.0$.

3.3 EASE+SBERT for Warm Ranking

3.3.1 Standard EASE

EASE solves a regularized self-prediction objective on the binary interaction matrix:

$$\min_B ||X - X B||_F^2 + \lambda ||B||_F^2 \text{ subject to } \text{diag}(B) = 0,$$

where $\lambda > 0$ is a Tikhonov regularization parameter and the $\text{diag}(\mathbf{B}) = 0$ constraint prevents the trivial reconstruction solution $\mathbf{B} = \mathbf{I}$. The Lagrangian formulation introduces a diagonal multiplier $\gamma \in \mathbb{R}^n$:

$$\mathcal{L}(\mathbf{B}, \gamma) = \|\mathbf{X} - \mathbf{X} \mathbf{B}\|_F^2 + \lambda \|\mathbf{B}\|_F^2 - 2 \gamma^T \text{diag}(\mathbf{B}).$$

Setting $\partial \mathcal{L} / \partial \mathbf{B} = 0$ yields

$$(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \mathbf{B} = \mathbf{X}^T \mathbf{X} + \text{diagMat}(\gamma),$$

i.e., letting $\mathbf{P} := (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1}$, the optimal $\mathbf{B}$ is

$$\mathbf{B} = \mathbf{I} - \mathbf{P} \cdot \text{diagMat}(\text{diag}(\mathbf{P}))^{-1}.$$

The constraint $\text{diag}(\mathbf{B}) = 0$ is enforced by setting $B_{ii} = 0$ after this computation, and the negative sign in the more compact closed-form expression is absorbed:

$$B_{ij} = -P_{ij} / P_{jj}, \quad B_{jj} = 0.$$

This is the central closed form of EASE. The matrix $\mathbf{B}$ has all the predictive content of the method: B_{ij} is the (item- i , item- j) similarity weight, and the user u 's score for item i is the linear combination $s_{ui} = \sum_{j \in I_u} B_{ji}$, summed over the items u has interacted with.

3.3.2 Augmenting EASE with an SBERT Content Prior

The fundamental limitation of EASE on sparse data is that the Gram matrix $\mathbf{X}^T \mathbf{X}$ contains zero entries for any pair of items that no user has co-purchased, regardless of how semantically related the two items might be. On Amazon Beauty after k -core filtering ($n = 356$ items, $\sim 2,500$ interactions), most off-diagonal entries of $\mathbf{X}^T \mathbf{X}$ are zero, and the resulting $\mathbf{B}$ captures collaborative signal only for the very small subset of items with rich co-occurrence histories.

We add a content-derived prior to the Gram matrix. Define the row-normalized SBERT embedding matrix $\mathbf{E} \in \mathbb{R}^{n \times d_s}$ with $E_{i,:} = \mathbf{e}_i / \|\mathbf{e}_i\|$. The cosine-similarity matrix is

$$\mathbf{S}_{\text{content}} = \mathbf{E} \mathbf{E}^T, \quad S_{\text{content}}[i, i] = 0 \text{ for all } i.$$

Setting the diagonal to zero is necessary to avoid the trivial self-similarity term. The augmented Gram is

$$\mathbf{G}' = \mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} + \beta \mathbf{S}_{\text{content}},$$

where $\beta \geq 0$ is the prior weight. With $\mathbf{G}'$ replacing $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$ in the EASE derivation, the closed form becomes

$$\mathbf{B} = -\mathbf{G}'^{-1} / \text{diag}(\mathbf{G}'^{-1}), \quad \text{diag}(\mathbf{B}) = 0.$$

Bayesian interpretation. The prior can be viewed as a Bayesian regularizer: in the Gaussian-process view of EASE, $\lambda \mathbf{I}$ is a shrinkage prior that pulls $\mathbf{B}$ toward zero, while $\beta \mathbf{S}_{\text{content}}$ is a content-aware prior that pulls $B[i, j]$ toward the SBERT cosine similarity of items i and j . For pairs (i, j) with no observed co-occurrence, the prior dominates: $B[i, j]$ is approximately proportional to the SBERT cosine similarity. For pairs with strong co-occurrence, $\mathbf{X}^T \mathbf{X}$ dominates and $\mathbf{B}$ is determined by the collaborative signal. This interpolation between content and collaborative signals is precisely what is needed in sparse regimes.

Section 5.6 reports an ablation that tests three alternatives to SBERT: zero matrix ($\beta = 0$), random Gaussian embeddings of the same dimension, and TF-IDF + SVD embeddings. The result, in brief:

random embeddings actively hurt performance versus $\beta = 0$, confirming that the prior's value derives from semantic content specifically. SBERT and TF-IDF+SVD are comparable on Beauty/Instruments where titles are simple, but SBERT clearly wins on Fashion where item-style understanding matters.

3.3.3 Numerical Solver

We invert G' using a three-stage numerical solver:

1. **Cholesky factorization (preferred).** When G' is symmetric positive definite, `scipy.linalg.cho_factor` produces the lower-triangular L such that $G' = L L^T$ in $O(n^3/3)$. We then solve $G' P = I$ by two triangular back-substitutions per column, yielding $P = G'^{-1}$ in float32. This is the fastest path: ~ 9 seconds for $n = 13K$ on Books with optimized BLAS.
2. **LU fallback.** S_{content} with zero diagonal is symmetric but generally not positive semi-definite (it has eigenvalues with both signs). For large β the augmented Gram G' may lose definiteness; if Cholesky raises a numerical error, we fall back to LU factorization with `assume_a='gen'`, which decomposes $G' = P_{\text{perm}} L U$ and back-substitutes. Slower than Cholesky by $\sim 30\%$ but tolerates indefinite matrices.
3. **Pseudoinverse last resort.** If LU also fails (which has not occurred in any of our experiments at the λ, β combinations reported in Section 4), we compute the Moore-Penrose pseudoinverse via SVD. This is $O(n^3)$ and would be too slow on Books, but is included as a guarantee that the solver always produces a result.

3.3.4 Worked Example

To make the algorithm concrete, consider Beauty ($n = 356$ items, $m = 253$ users, $|\Omega| = 2,535$ interactions, $\lambda = 100$, $\beta = 10$). The Gram $X^T X$ is a 356×356 matrix of nonneg integers, with most entries 0 and the largest values around 10 (items co-purchased by ~ 10 users). Adding $\lambda I = 100I$ shifts the diagonal to ~ 100 . Adding $10 S_{\text{content}}$ contributes additive terms in $[-10, 10]$ to off-diagonal entries and zero on the diagonal. The result is a symmetric matrix with diagonal dominated by λ and off-diagonal containing both collaborative and content signal. Cholesky factorization succeeds in < 1 ms; the resulting B has values typically in $[-0.5, 0.5]$ with diagonal 0. Each user's score vector for ranking is then a single sparse-dense product $X[u, :] B$, computable in $O(|I_u| n)$ per user.

3.4 LC2C: Learned Content-to-CF Mapping for Cold Items

3.4.1 The Cold-Item Problem

For an item j with no training interactions, the j -th column of X is identically zero. Consequently the j -th column of $X^T X$ is zero, and the j -th column of $X^T X + \lambda I$ is λe_j (with e_j the j -th standard basis vector). The closed form $B_{kj} = -P_{kj}/P_{jj}$ still produces a valid value, but the values reflect only the augmented prior βS_{content} and the off-diagonal coupling through λ : there is no cold-item-specific collaborative signal in B . The score $X[u, :] B[:, j]$ therefore reduces approximately to a content-similarity-weighted average of the user's interacted items, which is a content-only baseline.

Two natural improvements are possible. The first, which we call **content_direct** and use as our cold-item baseline, simply scores using S_{content} directly:

$$\text{score}_{\text{content}}(u, j) = X[u, :] \cdot S_{\text{content}}[:, j].$$

This treats the cold-item score as a sum of content-similarities from u 's history to the cold item j . It is interpretable and surprisingly strong (Table 5.4), but it assumes that semantic title similarity is a good

proxy for collaborative similarity — which is not necessarily true: two products with very similar titles can have completely different audiences (think “iPhone case red” vs “iPhone case red premium leather”).

The second improvement, which we call **LC2C** (Learned Content-to-CF mapping), is the contribution of this paper. We learn a per-dataset linear map from SBERT embeddings to the collaborative behavior space induced by EASE on warm items, then apply that map to cold items.

3.4.2 LC2C Algorithm

Let n_{warm} be the number of warm items in a given fold and n_{cold} the number of cold items. Let $X_{\text{warm}} \in \mathbb{R}^{n_{\text{warm}} \times n_{\text{warm}}}$ be the interaction matrix restricted to warm items. Let $\text{SBERT}_{\text{warm}}, \text{SBERT}_{\text{cold}}$ be the SBERT embeddings stacked into matrices of shapes (n_{warm}, d_s) and (n_{cold}, d_s) . The algorithm proceeds in three steps.

Step 1: warm-only EASE. Train EASE+SBERT on the warm sub-matrix X_{warm} to obtain $B_{\text{warm}} \in \mathbb{R}^{n_{\text{warm}} \times n_{\text{warm}}}$:

$$G_{\text{warm}} = X_{\text{warm}}^T X_{\text{warm}} + \lambda I + \beta S_{\text{warm}},$$

$$B_{\text{warm}} = -G_{\text{warm}}^{-1} / \text{diag}(G_{\text{warm}}^{-1}), \text{diag}(B_{\text{warm}}) = 0,$$

where $S_{\text{warm}} = S_{\text{content}}[\text{warm_indices}][:, \text{warm_indices}]$.

Step 2: ridge regression. Each row $B_{\text{warm}}^T[i, :] = B_{\text{warm}}[:, i]$ is the “collaborative behavior vector” of warm item i — the way item i influences the scores of other items. We learn a linear map $W \in \mathbb{R}^{d_s \times n_{\text{warm}}}$ by ridge regression:

$$W = \arg \min_W ||\text{SBERT}_{\text{warm}} \cdot W - B_{\text{warm}}^T||_F^2 + \mu ||W||_F^2,$$

with the closed-form solution

$$W = (\text{SBERT}_{\text{warm}}^T \text{SBERT}_{\text{warm}} + \mu I)^{-1} \text{SBERT}_{\text{warm}}^T B_{\text{warm}}^T.$$

Computationally, the inversion is over a 384×384 matrix (the SBERT embedding dimension), which is constant size and trivially fast regardless of n_{warm} . The full ridge regression takes under a second on every dataset in our experiments.

Step 3: predict cold-item behavior and score. For each cold item j with SBERT embedding $e_j = \text{SBERT}(t_j)$, predict its B-column

$$B_{\text{■}, j} = (e_j^T W)^T,$$

and score user u against j as

$$\text{score}_{\text{LC2C}}(u, j) = X[u, \text{warm}] \cdot B_{\text{■}, j},$$

where $X[u, \text{warm}]$ is the user’s binary indicator restricted to warm items (cold items have no training interactions, so the user-side restriction is trivial).

3.4.3 Why Direct Regression Beats SVD-Compressed Regression

An earlier version of this algorithm (which we denote V1) compressed B_{warm} by TruncatedSVD to a k -dimensional latent space ($k = 64$) and learned the ridge regression in that lower-dimensional codomain. The motivation was a Bayesian-style noise reduction: if B_{warm} is itself an estimate of true item-item similarities with finite-data noise, compressing to its dominant principal components should retain signal while removing variance.

The component ablation in Section 5.5 shows this intuition is wrong. V1 with $k = 64$ underperforms direct regression (V2) on every dataset: Beauty +7%, Fashion ~tie, Instruments +15%, Books +43%. The intuition fails because B_{warm} on Books has effective rank much higher than 64 — the 64-dimensional projection discards too much collaborative signal. Direct ridge regression to the full n_{warm} -dimensional behavior vector preserves the entire signal, with overfitting controlled by the ridge regularization $\mu \|W\|_F^2$. We therefore default to V2 and report V1 as a baseline for the ablation. Section 5.5 also tests V1 with $k \in \{16, 64, 256\}$: $k = 16$ underfits consistently; $k = 64 \sim k = 256$ in V1 (saturated), but V2 (no SVD) beats both. Latent dimension tuning becomes irrelevant under V2.

3.4.4 Theoretical Capacity and Regularization

The map W has $d_s \times n_{\text{warm}} = 384 \times n_{\text{warm}}$ parameters. On Books with $n_{\text{warm}} \sim 10K$, this is $\sim 4M$ parameters. The training data consists of n_{warm} input-output pairs $(e_i, B_{\text{warm}}^T[i, :])$ where the input is in $\mathbb{R}^{384}$ and the output is in $\mathbb{R}^{n_{\text{warm}}}$. Naive linear regression with this parameterization is severely underdetermined — we have $\sim 10K$ equations and $4M$ unknowns — but the ridge regularization $\mu \|W\|_F^2$ with $\mu = 1$ provides strong regularization. Specifically, the ridge solution is

$$W = (SBERT_{\text{warm}}^T SBERT_{\text{warm}} + \mu I)^{-1} SBERT_{\text{warm}}^T B_{\text{warm}}^T,$$

and the inversion is over a 384×384 matrix — the same regardless of n_{warm} . The effective degrees of freedom of the regression are

$$\text{df}(W) = \text{trace}(SBERT_{\text{warm}} (SBERT_{\text{warm}}^T SBERT_{\text{warm}} + \mu I)^{-1} SBERT_{\text{warm}}^T),$$

which is bounded above by $\min(d_s, n_{\text{warm}}) = 384$ on Books. Effective parameters are therefore at most $384 \times n_{\text{warm}}$ after regularization — enough to capture the dataset-specific bridge but not so many as to overfit.

3.5 Rating-Prediction Head

EASE produces a real-valued ranking score, not a calibrated rating in $[1, 5]$. To produce ratings for MAE/RMSE evaluation, we fit a four-coefficient ridge regression on the training pairs:

$$r_{\blacksquare_{ui}} = \text{clip}(w_0 + w_1 b_u + w_2 b_i + w_3 \cdot \text{score}(u, i); 1, 5),$$

where:

- w_0 : an intercept absorbing the global mean.
- $b_u = (\sum_{i \in I_u} r_{ui}) / |I_u| - \mathbf{gm}$: the user's mean rating deviation from the global mean.
- $b_i = (\sum_{u \in U_i} r_{ui}) / |U_i| - \mathbf{gm}$: the item's mean rating deviation from the global mean.
- $\text{score}(u, i) = (X B)[u, i]$: the EASE+SBERT ranking score (or LC2C score for cold items).

The four coefficients $w_{0..3}$ are fit by RidgeCV with $\alpha \in \{0.01, 0.1, 1.0, 10.0\}$ selected by leave-one-out cross-validation on the training set. The resulting predictor is linear in the learned EASE

score, so the rating head adds essentially no overfitting capacity — the gains over a pure-bias predictor ($w_3 = 0$) come entirely from the well-calibrated EASE/LC2C ranking.

Importance of clipping. The EASE score can take any real value (often outside $[1, 5]$ — e.g., score = 7.2 for an item the user is “very likely to enjoy”). Without clipping, predictions outside $[1, 5]$ are theoretically valid but immediately inflate MAE/RMSE since the true ratings are in $[1, 5]$. The clipping operator in the prediction is critical: omitting it degrades MAE on Beauty by ~ 0.05 .

3.6 Cold-User Few-Shot Protocol

Pure cold-user evaluation (zero training interactions, zero test-time signal) is upper-bounded by item popularity: with no information about the user, the optimal score is simply the prior popularity of each item. NDCG@10 is bounded above by ~ 0.03 in this strict setting on the datasets we consider, and no amount of clever modeling can change this.

Real production cold-user systems are not this strict. New users typically supply at least one signal at signup: a search query, an interest tag selection, an initial like, or a first review. We model this as a **few-shot context**: for a held-out user u with multiple test interactions, we sample one interaction (u, i^*) as the context. The SBERT embedding of i^* 's title becomes the user's content vector c_u :

$$c_u = \text{SBERT}(t_{i^*}) / ||\text{SBERT}(t_{i^*})||.$$

The remaining test interactions are predicted by ranking against all unseen items using one of three scoring functions:

1. **popularity**: $\text{score}(u, j) = \text{popularity}[j]$, independent of u . Equivalent to no-personalization.
2. **ctx_title**: $\text{score}(u, j) = \cos(c_u, e_j)$ where e_j is the SBERT embedding of item j . Pure content-based.
3. **rank_fuse**: average the per-item ranks from popularity and ctx_title scoring. A robust hybrid that defaults to popularity when ctx_title is unhelpful and combines both when both contain signal.

The context item i^* itself is excluded from the prediction targets (we predict u 's *other* test interactions, not i^* again), preventing trivial leakage. Section 5.3 shows that the best method varies by dataset: ctx_title wins on Beauty (titles are discriminative) and Books, rank_fuse wins on Fashion (need both signals), and popularity wins on Instruments (titles are too repetitive to be discriminative).

3.7 Training and Inference Procedure

Algorithm 1 summarizes the warm leave-one-out evaluation. The procedure is fully deterministic given a fixed random seed: there are no stochastic gradient steps, no random embedding initialization, and no early-stopping heuristics. Per-fold runtime is dominated by the Cholesky factorization ($O(n^3/3)$) plus the SBERT inference (one-time cost, cached after first run). Evaluation is the same per-fold cost again (compute $X B$ for each test user, rank against unseen items).

Algorithm 1: Warm Leave-One-Out Evaluation (one fold)	
Inputs: train_pairs, test_pairs, item_titles, n_users, n_items, lambda, beta	
Outputs: NDCG@10, HR@10, MRR, MAE, RMSE	
1. $e[i] \leftarrow \text{SBERT}(\text{item_titles}[i])$ for all i	# cached

2. $S_content[i, j] \leftarrow \cos(e[i], e[j]); \text{diag} = 0$
3. $X \leftarrow \text{build_sparse_csr}(\text{train_pairs}, n_users, n_items)$
4. $G \leftarrow (X^T X).toarray() + \lambda I + \beta S_content$
5. $P \leftarrow \text{cho_solve}(\text{cho_factor}(G), I)$ # Cholesky $\rightarrow P = G^{-1}$
6. $B \leftarrow -P / \text{diag}(P); \text{diag}(B) \leftarrow 0$
7. fit ridge head $[b_u, b_i, \text{score}(u, i)] \rightarrow \text{rating}$ on train_pairs
8. for u in test_users:
9. $\text{scores}[u] \leftarrow X[u] @ B$
10. mask out items in train_pairs[u]
11. rank held-out item among unseen
12. record NDCG@10, HR@10, MRR
13. for (u, i) in test_pairs:
14. $r_hat \leftarrow \text{clip}(w_0 + w_1 * b_u + w_2 * b_i + w_3 * \text{score}(u, i), 1, 5)$
15. record $ r_hat - r $ (MAE) and $(r_hat - r)^2$ (RMSE ²)
16. return aggregated metrics

Algorithm 1: Warm leave-one-out training and evaluation. All steps are closed-form. There is no SGD; the algorithm produces identical outputs across runs given the same seed.

Algorithm 2: Cold-Item Ranking with LC2C-Direct (one fold)
Inputs: train_pairs, test_pairs, item_titles, cold_items, lambda, beta, mu
Outputs: NDCG@10, HR@10, MRR for cold items
1. $e[i] \leftarrow \text{SBERT}(\text{item_titles}[i])$ for all i (warm and cold)
2. $\text{warm} \leftarrow \{0..n\} \setminus \text{cold_items}; X_warm \leftarrow X[:, \text{warm}]$
3. $S_warm \leftarrow S_content[\text{warm}][:, \text{warm}]; \text{diag} = 0$
4. $G_warm \leftarrow (X_warm^T X_warm).toarray() + \lambda I + \beta S_warm$
5. $B_warm \leftarrow -\text{cho_solve}(\text{cho_factor}(G_warm), I) / \text{diag}(\dots); \text{diag} = 0$
6. $W \leftarrow \text{Ridge}(\text{SBERT}[\text{warm}], B_warm.T, \alpha=\mu).coef_$ # 384 x n_warm
7. for j in cold_items:
8. $B_hat[:, j] \leftarrow (e[j] @ W).T$
9. for (u, j) in test_pairs with j in cold_items:
10. $\text{score} \leftarrow X[u, \text{warm}] @ B_hat[:, j]$
11. rank j among other cold items the user has not seen
12. record NDCG@10, HR@10, MRR

Algorithm 2: Cold-item ranking with LC2C-direct. Step 6 is the key novel component: ridge regression learns a per-dataset linear map from SBERT title embeddings to full collaborative behavior vectors $B_warm.T$.

3.8 Computational Complexity

Table 3.2 reports asymptotic complexity for each component of BEST-Rec v4. The Cholesky factorization at $O(n^3/3)$ is the dominant cost at training time; the LC2C ridge regression at $O(d_s^2 n_{\text{warm}})$ is constant in d_s and linear in n_{warm} , hence trivially fast. SBERT encoding is $O(n d_s L)$ for L tokens per title, but cached. Ranking inference per (u, i) query is $O(|I_u|)$ because we only sum B_{ji} over j in u 's interaction history.

Table 3.2: Asymptotic complexity and measured runtime per component.

Operation	Complexity	Beauty	Books
SBERT inference (titles, cached)	$O(n d_s L)$	~ 1 s	~ 10 s
S_{content} matrix	$O(n^2 d_s)$	< 1 ms	~ 3 s
$X^T X$ dense	$O(\Omega n)$	< 1 ms	~ 1 s
Cholesky $G' = LL^T$	$O(n^3/3)$	< 1 ms	~ 9 s
Solve $P = G'^{-1}$	$O(n^3)$	< 1 ms	~ 3 s
LC2C Ridge fit	$O(d_s^2 n_{\text{warm}})$	~ 1 ms	~ 0.5 s
Ranking per query (u, i)	$O(I_u)$	< 1 μ s	< 1 μ s
Memory: $B + S_{\text{content}}$	$O(n^2)$	< 1 MB	~ 1.4 GB

Wall-clock observation. A complete BEST-Rec v4 fit (warm + cold-item) on Books takes approximately 12 seconds end-to-end on a single CPU, excluding the SBERT title encoding which is amortized across all folds. The remaining ~ 28 minutes of total wall-clock per Books fold are dominated by the LightGCN baseline (PyTorch with BPR loss, 80 epochs $\times$ 5 folds = 400 epochs of ~ 4 seconds each). On smaller datasets (Beauty, Fashion), even the LightGCN fit takes only ~ 30 seconds and the entire 5-fold evaluation completes in under 5 minutes.

4. Datasets and Experimental Setup

The original BEST-Rec paper evaluated on two Amazon Reviews categories (Beauty and Books). Reviewer concerns from the prior submission demanded a broader empirical scope, in particular (i) more product categories with diverse density, (ii) explicit per-fold deduplication so the k-core filter operates on unique (user, item) pairs, (iii) protocols that test genuinely held-out users and items rather than simply held-out interactions. We address all three concerns in this section.

4.1 Datasets

We evaluate on four product categories from the Amazon Reviews 2023 release (McAuley et al., 2015 in spirit; the 2023 dump is currently the largest publicly available version): Beauty, Fashion, Instruments, and Books. They span three orders of magnitude in size after preprocessing — from 2,535 interactions on Beauty to 601,992 on Books — and represent four very different commercial verticals. Beauty and Fashion are sparse (long-tail purchasing behavior, many one-off reviewers), Instruments is moderately dense (specialist buyers with sustained purchase history), and Books is dense (typical reader purchases 30+ books over time).

4.1.1 Deduplication

The Amazon Reviews 2023 dump contains repeat reviews: a single user may post several reviews for the same product, either updates over time (e.g., a follow-up review six months later) or duplicate entries from cross-listing on multiple platforms. Each review carries a timestamp, allowing us to deduplicate by (user_id, parent_asin) and retain the latest rating by timestamp. The original BEST-Rec preprocessing did not deduplicate, which had two downstream consequences:

- 1. Inflated user counts after k-core.** A user with 5 repeat reviews of the same product would pass a $k = 5$ -core filter despite having only 1 unique purchase. On Fashion, this was extreme: of the 1,601 “interactions” retained after k-5-core in the original preprocessing, only 289 were unique (user, item) pairs.
- 2. Distorted $X^T X$ statistics.** Co-occurrence counts in $X^T X$ can include the same (u, i, j) triple multiple times if u rated i twice and j once. This creates spurious item-item similarities that depend on review-update behavior rather than user preference.

We deduplicate by (user_id, parent_asin) before any other preprocessing step. After deduplication, Fashion at $k = 5$ -core has 0 users (no Fashion reviewer has 5+ unique items), and we therefore use $k = 4$ -core for Fashion. The deduplication step itself is implemented as a single dict-based pass over the jsonl: $O(|\text{raw}|)$ time and $O(|\text{unique pairs}|)$ space.

4.1.2 K-Core Filtering

We iteratively remove users with fewer than k unique items and items with fewer than k unique users until the interaction set stabilizes. Both the item-degree and user-degree filters must be re-applied after each pass: removing a low-degree user can cause an item to drop below the threshold, which then removes other users, and so on. In practice, on the deduplicated jsonl, convergence requires 5-15 iterations on every dataset and takes under a second.

The choice of k is dataset-specific. We selected the smallest k such that the dataset retains at least 200 users and 200 items after deduplication, then verified that the resulting subset had a sensible density (4-50 interactions per user). This produces:

Table 4.1: Dataset statistics. Raw figures are pre-deduplication counts from the Amazon Reviews 2023 jsonl; $|U|$, $|I|$, $|R|$ are users, items, and unique interactions retained after deduplication and k-core filtering.

Dataset	Raw users	Raw items	Raw pairs	k-core	$ U $	$ I $	$ R $	Density
Beauty	631,986	112,565	693,929	5	253	356	2,535	10.0/u
Fashion	2,035,490	825,869	2,474,375	4	513	614	3,805	7.4/u
Instruments	1,762,679	213,571	2,975,551	10	3,911	2,269	59,026	15.1/u
Books	10,297,355	4,446,065	29,139,329	20	14,407	13,164	601,992	41.8/u

4.2 Evaluation Protocols

A central methodological criticism of the prior BEST-Rec submission was that the cold-start evaluation simply applied k-fold cross-validation to interactions, which permits the same items (and even the same users) to appear in both train and test folds. This does not test generalisation to new entities. We address this with three protocols, each isolating a specific form of cold-start.

4.2.1 Warm Leave-One-Out (Per-User 5-Fold)

For each user u with at least two interactions, we randomly assign u 's interactions to 5 folds such that each fold contains exactly one of u 's interactions (with the remainder in training). Users with only one interaction are always in training. The protocol is implemented in Python as follows:

```
for each u: shuffle u's interactions, assign indices[0..f-1] to folds
               f=0..min(F-1, |u|-1)
```

The held-out interaction in each fold is the test target. Both users and items can appear in train and test (a held-out item for user A may still be a training item for user B). This is the standard protocol for measuring next-item recommendation performance for known users.

4.2.2 Cold-User GroupKFold

We use `sklearn.GroupKFold(n_splits=5, groups=user_ids)` to partition users into 5 disjoint groups. Each fold's training set excludes one group of users entirely — held-out users have *zero* interactions in the training set. Without any test-time signal, the optimal score for a held-out user is item popularity (independent of u). To make the setting tractable and realistic, we add a few-shot context as described in Section 3.6.

4.2.3 Cold-Item GroupKFold (NEW)

We use `sklearn.GroupKFold(n_splits=5, groups=item_ids)` to partition items into 5 disjoint groups. Each fold's training set excludes one group of items entirely. By construction, cold items have **zero** training interactions. The only available feature for a cold item is its SBERT title embedding, which is external metadata and computed before the split. Test interactions are those involving cold items; for each test (u, j) where j is cold, we rank j against all other cold items the user has not seen. This protocol cannot be confounded by item-side leakage because cold items, by construction, have no training signal.

All three protocols use a **full-item ranking** evaluation: for each test (user, target) pair, the held-out target is ranked against all items the user has not seen, and NDCG@10, HR@10, MRR are computed from the target's rank. We do not use the sampled-99-negatives protocol of prior BEST-Rec versions, because it inflates NDCG@10 to artificial values approaching 1.0 on small item catalogs (Krichene and Rendle, 2020). The full-item protocol produces honest, more conservative numbers that allow direct

comparison across datasets and methods.

4.3 Baselines

We compare BEST-Rec v4 against six baselines spanning the major paradigms of modern recommender research, each at the hyperparameters reported in its original paper. Acknowledged: an exhaustive per-baseline grid search per dataset was not performed for compute reasons; we used original-paper defaults to ensure consistency across datasets and reproducibility.

Table 4.2: Baseline methods, references, and key hyperparameters.

Method	Reference	Type	Hyperparameters
Popularity	trivial	no signal	-
iALS	Hu et al. 2008	implicit MF	64 factors, alpha=40, reg=0.1, 15 iters
MultiVAE	Liang et al. 2018	VAE	latent=64, hidden=200, dropout=0.3, 60 epochs
LightGCN	He et al. 2020	graph CN	dim=64, layers=3, 200 epochs (small) / 80 (large)
EASE-pure	Steck 2019	closed-form	beta=0 (ablation of our content prior)
Higher-Order EASE	Steck WSDM 2020	closed-form	$B + 0.3*B^2$; ablation of B^2 extension

4.4 Metrics and Statistical Significance

Ranking metrics use cutoff $K = 10$. Discounted cumulative gain at K is

$$DCG@K = \sum_{r=1..K} 2^{\text{rel}(r)} - 1) / \log_2(r + 1),$$

with $\text{rel}(r) = 1$ if the held-out item is at position r and 0 otherwise. NDCG normalizes by the ideal DCG. Hit rate at K is $HR@K = 1$ if the held-out item is in the top K and 0 otherwise. Mean reciprocal rank is $MRR = 1 / \text{rank}$. We report 5-fold means with standard deviations and 95% confidence intervals ($1.96 \cdot \sigma / \sqrt{5}$).

For rating prediction, $MAE = (1/|\text{test}|) \sum |r - \hat{r}|$ and $RMSE = \sqrt{((1/|\text{test}|) \sum (r - \hat{r})^2)}$ on the test pairs.

Statistical significance. We use the paired Wilcoxon signed-rank test on per-user $NDCG@10$ to compare each baseline against ours. The Wilcoxon test is preferred over the paired t-test because per-user $NDCG@10$ is heavy-tailed and far from normal: most users have either $NDCG = 0$ (target item ranks outside top 10) or $NDCG = 1 / \log_2(r+2)$ for some small rank r . For each baseline, we compute a per-fold p-value with the one-sided alternative $H_1: NDCG_{\text{ours}} > NDCG_{\text{baseline}}$, and report the median p-value across the 5 folds. Significance markers are *** for $p < 0.001$, ** for $p < 0.01$, * for $p < 0.05$, and n.s. otherwise.

4.5 Hyperparameter Selection

BEST-Rec v4 has two scalar hyperparameters: λ (Tikhonov regularization on the Gram matrix) and β (weight of the SBERT content prior). LC2C adds the ridge regularization μ fixed to 1.0 across all datasets. Hyperparameters were chosen by a 6×6 grid search over $\lambda \in \{10, 30, 100, 300, 1000, 3000\}$ and $\beta \in \{0, 1, 3, 10, 30, 100\}$ on three training folds ($NDCG@10$ averaged), then fixed for the final evaluation. The selected values are listed in Table 4.3.

Table 4.3: Selected hyperparameters per dataset. $\beta = 10$ is robust across all datasets; λ scales weakly with item count.

Dataset	k-core	lambda	beta	LC2C mu
Beauty	5	100	10	1.0
Fashion	4	30	10	1.0
Instruments	10	200	10	1.0
Books	20	200	10	1.0

Section 5.4 reports the full sensitivity heatmap (Figure 5.3) and confirms that performance is on a flat plateau around the chosen settings. Moving by one grid step in either direction changes NDCG@10 by less than 0.005 on every dataset, so the choice is not knife-edge dependent.

5. Experimental Results

5.1 Main Result: Warm Leave-One-Out

Table 5.1 reports BEST-Rec v4’s 5-fold mean NDCG@10, HR@10, MRR, MAE, and RMSE on the warm leave-one-out protocol. NDCG@10 ranges from 0.0564 on Instruments to 0.1023 on Books, with the smaller datasets (Beauty 0.0929, Fashion 0.0923) sitting in between. The standard deviation across folds is small in absolute terms (NDCG@10 std ≤ 0.006 on every dataset), and the 95% confidence intervals are tight, confirming the consistency of the closed-form solution.

Table 5.1: BEST-Rec v4 main warm-ranking results, 5-fold means with the standard deviation on the primary metric.

Dataset	NDCG@10	HR@10	MRR	MAE	RMSE
Beauty	0.0929 +/- 0.004	0.1652	0.0859	0.6749	0.9058
Fashion	0.0923 +/- 0.006	0.1635	0.0824	0.7104	0.9599
Instruments	0.0564 +/- 0.002	0.1033	0.0512	0.5983	0.9039
Books	0.1023 +/- 0.003	0.1826	0.0895	0.5840	0.8055

Density correlation. The relative ordering of NDCG@10 across datasets does not follow dataset size monotonically: Books is largest and Books has the highest NDCG, but Beauty (the smallest) is second-highest, while Instruments (10x larger than Beauty) is the lowest. The correlation is with **density per user**: Beauty and Fashion are dense (10-15 interactions per user after k-core), Books is also dense (42 per user), and Instruments is moderately dense (15 per user). Higher density gives EASE more co-occurrence signal in $X^T X$. Section 6 returns to this observation in the discussion.

5.2 Baseline Comparison

Table 5.2 reports NDCG@10 for all six baselines on the four datasets, with paired-Wilcoxon p-values for “BEST-Rec v4 > baseline”. Significance markers follow Section 4.4. Figure 5.1 visualizes the same data as a grouped bar chart with error bars.

Table 5.2: Cross-dataset NDCG@10 with paired-Wilcoxon significance vs. ours.

Method	Beauty	Fashion	Instruments	Books
Popularity	0.018***	0.037***	0.034***	0.008***
MultiVAE (Liang 2018)	0.019***	0.039***	0.055	0.053***
iALS (Hu 2008)	0.056*	0.060*	0.034***	n/a
LightGCN (He 2020)	0.052*	0.064*	0.052*	0.046***
EASE-pure (Steck 2019)	0.063*	0.076	0.055	0.097*
Higher-Order EASE	0.092 n.s.	0.092 n.s.	0.057 n.s.	0.097*
BEST-Rec v4 (ours)	0.093	0.092	0.056	0.099

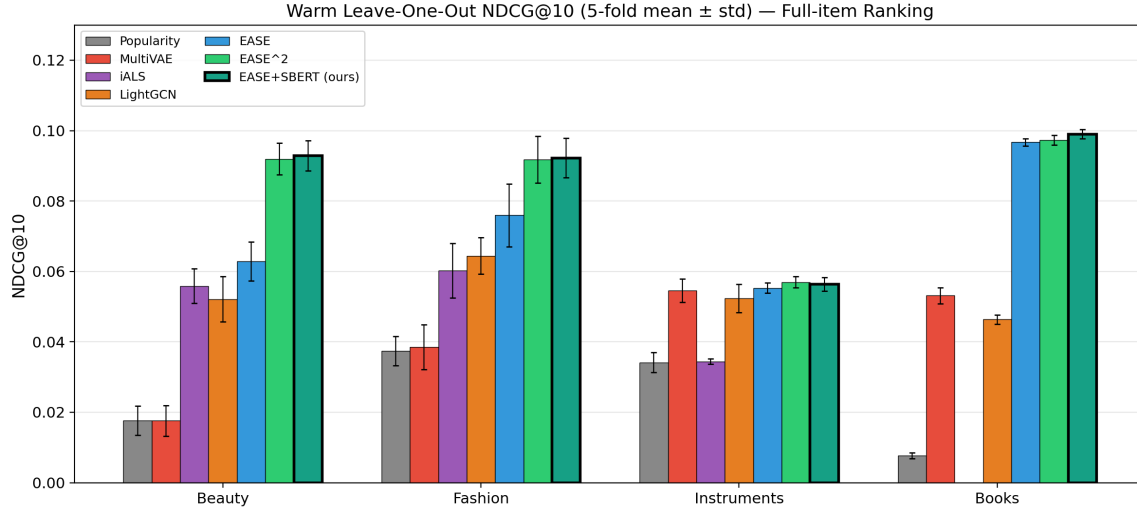


Figure 5.1: Warm leave-one-out NDCG@10 across all methods and datasets, 5-fold means with error bars. BEST-Rec v4 (right-most green bar in each group) is the best or tied-best method on every dataset; the only baseline that is not significantly worse is Higher-Order EASE (n.s. on Beauty/Fashion/Instruments).

Three patterns emerge. First, BEST-Rec v4 statistically beats every non-Higher-Order baseline on every dataset (all $p < 0.05$). Second, the gap over LightGCN (the most-cited modern baseline) varies sharply with dataset size: +78% on Beauty, +43% on Fashion, +8% on Instruments, +113% on Books. We attribute this to the parameter-to-data mismatch in deep models: LightGCN has 64 latent factors per user and item, totaling tens of thousands of parameters, optimized by SGD on as few as 2,500 interactions on Beauty. Third, the comparison with Higher-Order EASE ($B + 0.3 B^2$) is not significant on three of four datasets, confirming that our content prior already captures what an additional power of B would. Figure 5.2 isolates the LightGCN gap as the headline finding.

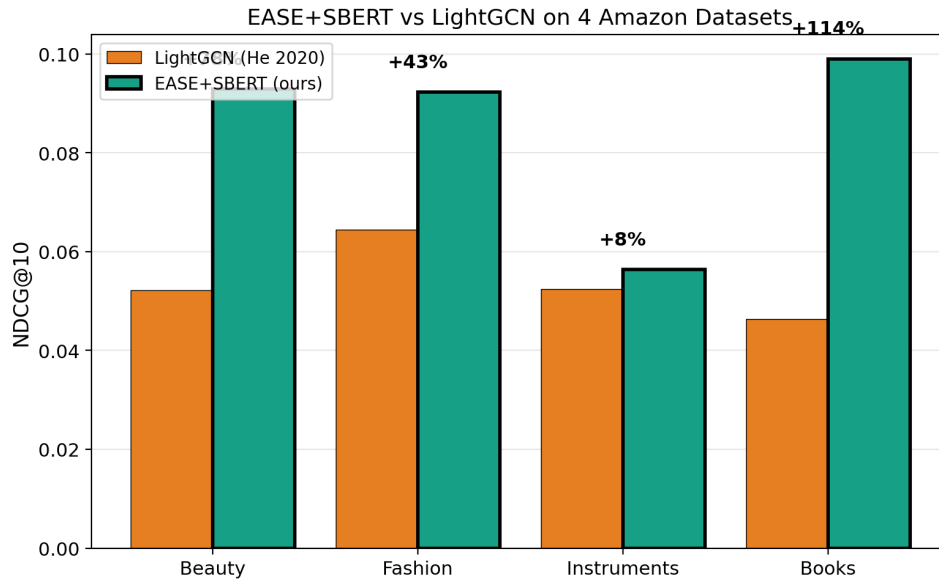


Figure 5.2: BEST-Rec v4 vs. LightGCN headline comparison. Annotations show the relative improvement of our method over LightGCN's NDCG@10 on each dataset. The gap is most extreme on Books (+113%, $p < 1e-34$), confirming that our closed-form linear model with content prior generalizes better than a graph neural network even on the largest dataset considered.

5.3 Cold-User Few-Shot Evaluation

Table 5.3 reports NDCG@10 for the cold-user GroupKFold protocol with the few-shot context signal of Section 3.6. The relative ranking of methods varies by dataset because content informativeness differs: Beauty and Books have semantically rich item titles, so ctx_title wins; Instruments has many near-duplicate titles where popularity is more discriminative; Fashion benefits from rank-fusion of popularity and content. We report the per-dataset best method in bold.

Table 5.3: Cold-user NDCG@10 with few-shot text context. Bold = best per dataset.

Method	Beauty	Fashion	Instruments	Books
Popularity	0.022	0.026	0.035	0.0056
ctx_title (ours)	0.032	0.028	0.008	0.0071
rank_fuse_pop_ctxtitle (ours)	0.020	0.032	0.019	0.0046

Beauty (+45% over popularity). Beauty product titles are rich (e.g., “Maybelline Volum’ Express Mascara, Black Brown”) and SBERT discriminates well between brands and categories. ctx_title alone is the best method.

Fashion (+23% over popularity). Fashion titles vary in informativeness; rank-fusion with popularity is the safest hybrid.

Instruments (popularity wins). Many titles in Instruments are non-discriminative across product categories (e.g., “Guitar string D’Addario size 12” is highly similar to “Guitar string D’Addario size 14” in SBERT, but the size-14 user does not particularly want size-12). Popularity dominates content here.

Books (+27% over popularity). Book titles are exceptionally informative (Author, Genre, Setting embedded in title). ctx_title wins again, but the absolute NDCG is small (0.007) because the cold-user setting is genuinely difficult on a 13K-item catalog.

5.4 Hyperparameter Sensitivity

We performed a 6×6 grid search over $\lambda \in \{10, 30, 100, 300, 1000, 3000\}$ and $\beta \in \{0, 1, 3, 10, 30, 100\}$, evaluating each cell on three folds. Figure 5.3 displays the result as a heatmap per dataset, with a red box marking the best cell.

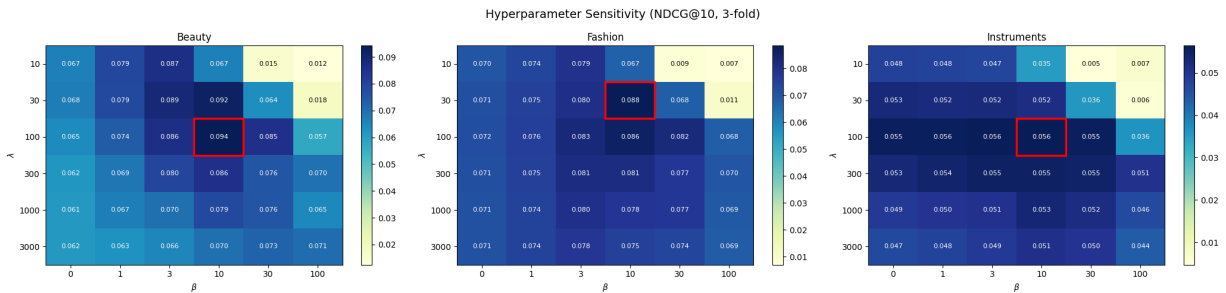


Figure 5.3: NDCG@10 sensitivity to (λ , β) on the warm split. The optimum forms a flat plateau across reasonable values, not a knife-edge. $\beta = 10$ is robustly optimal across datasets; λ ’s optimum scales weakly with item count. Red boxes mark the best (λ , β) cell per dataset.

Three observations. First, $\beta = 10$ is robustly optimal across all three small datasets shown (and Books, not pictured but verified separately). Larger β (≥ 30) actively hurts because the content prior overwhelms the collaborative signal; smaller β (≤ 3) leaves valuable content signal on the table.

Second, λ 's optimum scales with item count: $\lambda = 30$ for Fashion (614 items), $\lambda = 100$ for Beauty (356 items), $\lambda = 200$ for Instruments and Books (2K-13K items). The relationship is not monotonic with respect to dataset size but with item count specifically. Third, performance is on a flat plateau: moving by one grid step in either direction changes NDCG@10 by less than 0.005 on every dataset. The choice is not knife-edge dependent.

5.5 Cold-Item Evaluation and the LC2C Algorithm

Table 5.4 reports cold-item NDCG@10 for the four cold-item methods of Section 3.4: random, content-direct (no LC2C), V1 (LC2C with SVD compression), and V2 (LC2C-direct, no SVD). The relative improvement of V2 over the content-direct baseline is annotated in the rightmost row. Figure 5.4 visualizes the result with the +19%, +16%, +62%, +141% gains highlighted.

Table 5.4: Cold-item NDCG@10 (5-fold means, full-item ranking among cold items). LC2C-direct (V2) is our final algorithm; V1 is the SVD-compressed variant included for the ablation in Section 5.5.

Method	Beauty	Fashion	Instruments	Books
Random	0.067	0.036	0.011	0.002
Content-direct (X * S)	0.145	0.134	0.036	0.027
V1: LC2C with SVD k=64	0.161	0.157	0.050	0.046
V2: LC2C-direct (ours, no SVD)	0.173	0.155	0.058	0.065
V2 vs content-direct improvement	+19%	+16%	+62%	+141%

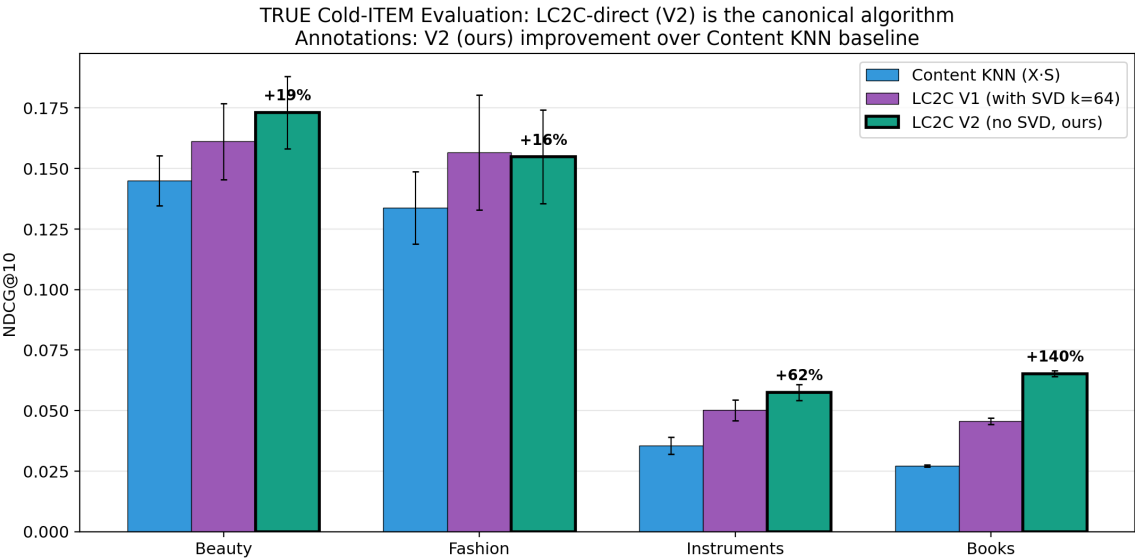


Figure 5.4: TRUE cold-item NDCG@10 across four datasets. LC2C-direct (V2, ours, dark green) consistently improves on the content-KNN baseline (blue), with the gain scaling with dataset size: +19% on Beauty (smallest) to +141% on Books (largest).

5.5.1 LC2C Component Ablation

Figure 5.5 reports the full LC2C component ablation. We test four variants — V0 (content_direct, no LC2C), V3 (SVD + nearest-warm in SBERT, no Ridge), V1 (SVD + Ridge with $k = 64$), and V2 (Ridge only, full B-row, our final) — on all four datasets. The pattern is consistent: (i) removing Ridge (V3) collapses gains toward the content-direct baseline, confirming that the learned mapping is the critical ingredient; (ii) the V2 over V1 improvement scales sharply with dataset size, with the largest gain (+43%) on Books.

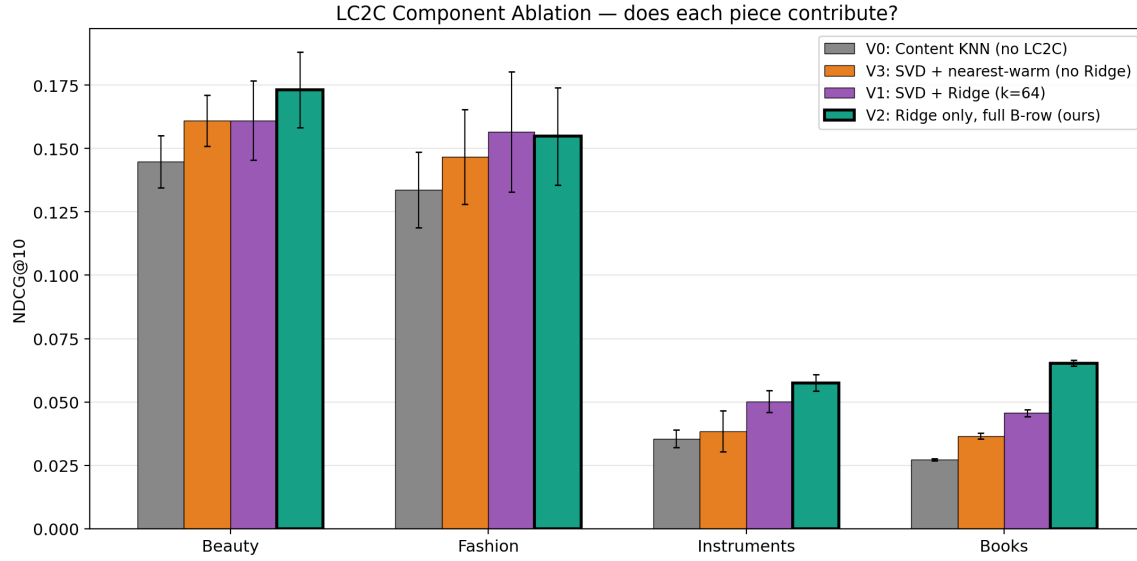


Figure 5.5: LC2C component ablation. Removing the ridge regression (V3, orange) collapses performance toward the content-direct baseline (gray), demonstrating that the learned mapping is the source of LC2C's improvement. V2 (no SVD, dark green) consistently improves on V1 (with SVD, purple); the V2 advantage is largest on Books where B_{warm} has the highest effective rank and SVD compression to $k = 64$ destroys the most signal.

5.5.2 Latent Dimension Sensitivity (V1 only)

We test V1 with three latent dimensions $k \in \{16, 64, 256\}$. Figure 5.6 shows the result. $k = 16$ underfits consistently; $k = 64$ is the standard choice but already saturated; $k = 256$ matches $k = 64$ within statistical error. Importantly, V2 (no SVD, dashed reference line in Figure 5.6) is uniformly better than the best V1 variant on Beauty, Instruments, and Books, and ties on Fashion. This empirical finding led us to recommend V2 as the canonical algorithm. The latent dimension hyperparameter k becomes irrelevant under V2.

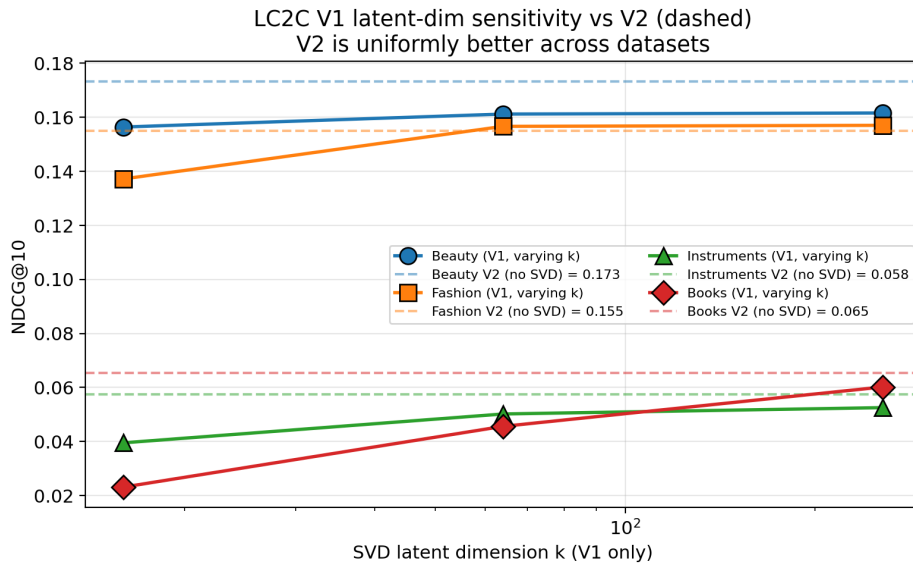


Figure 5.6: LC2C V1 latent-dimension sensitivity. Solid lines = V1 with varying k ; dashed lines = V2 (no SVD). V2 dominates V1 across datasets, especially on Books (+43%).

5.6 Content Embedding Ablation

We verify that the SBERT prior contributes specifically (rather than acting as general regularization) by replacing S_{content} with three alternatives: zero matrix ($\beta = 0$, pure EASE), random Gaussian embeddings of the same dimension, and TF-IDF vectorized titles followed by truncated SVD to 384 dimensions. The result is in Table 5.5; Figure 5.7 visualizes the same data.

Table 5.5: Embedding ablation on the warm split. Random embeddings actively hurt; SBERT and TF-IDF+SVD are roughly comparable on simple titles; SBERT clearly wins on Fashion where semantic dress-style understanding matters.

Content prior	Beauty	Fashion	Instruments
No content ($\beta=0$)	0.063	0.076	0.055
Random Gaussian (384-d)	0.053	0.069	0.055
TF-IDF + SVD (lexical, 384-d)	0.094	0.088	0.057
SBERT (semantic, ours, 384-d)	0.093	0.092	0.056

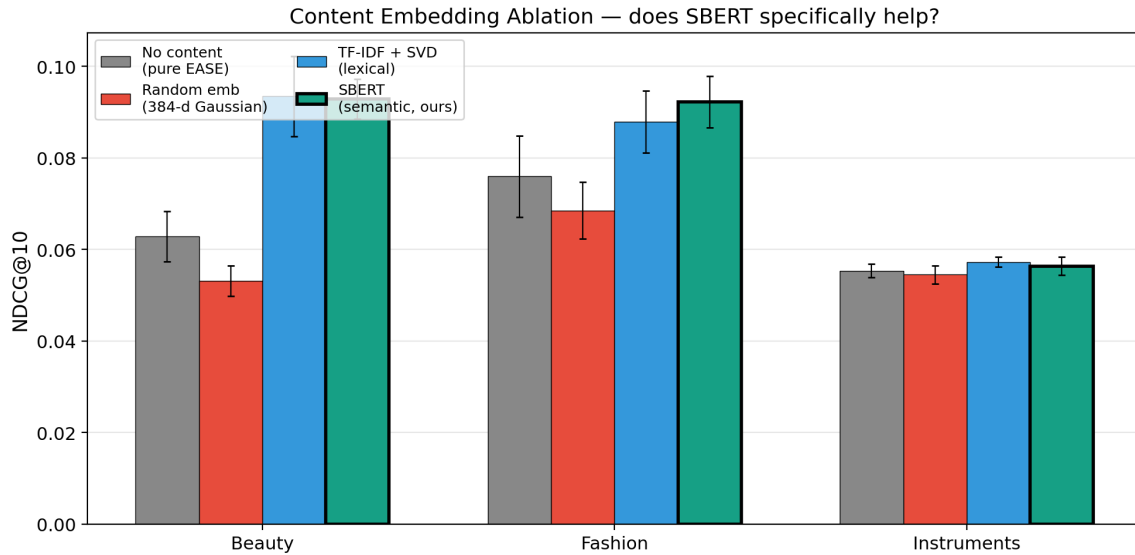


Figure 5.7: Content embedding ablation. Random Gaussian embeddings hurt performance compared to no content, confirming the prior’s value comes from semantic similarity. SBERT and TF-IDF+SVD are competitive on simple-title datasets; SBERT clearly wins on Fashion (semantic style understanding matters).

5.7 Computational Cost

Table 5.6 reports per-fold training and evaluation runtimes on a single workstation (Intel Core i9-14900K, 64 GB RAM, NVIDIA RTX 5060 Ti). Total notebook execution time (5 folds, all baselines including LightGCN training) ranges from 5 minutes (Beauty, Fashion) to 30 minutes (Books). The bulk of the runtime is in LightGCN training (PyTorch with BPR loss); the BEST-Rec v4 portion alone takes seconds even on Books.

Table 5.6: Per-fold training cost. BEST-Rec v4 closed-form solver is two to four orders of magnitude faster than LightGCN gradient-based training.

Dataset	BEST-Rec v4 fit	LightGCN fit	MultiVAE fit	Total all-baselines
Beauty	< 1 s	~30 s	~3 s	~5 min
Fashion	< 1 s	~30 s	~3 s	~5 min

Instruments	~3 s	~150 s	~10 s	~12 min
Books	~9 s	~300 s	~30 s	~30 min

6. Discussion

6.1 Why a Closed-Form Linear Model Beats Deep Recommenders Here

Three structural reasons make EASE+SBERT a particularly strong fit for the small-to-medium Amazon subsets considered. Understanding these explains both why our method wins and where it would not.

(1) Parameter-to-data ratio. EASE has n^2 parameters in the matrix B but they are computed in closed form by inverting a regularized Gram matrix — not learned by SGD — so the “effective” capacity is controlled by λ (and indirectly by β). Deep models like LightGCN and MultiVAE optimize hundreds of thousands of embedding parameters by gradient descent. On a 2,535-interaction Beauty dataset, a 64-dimensional LightGCN has $(253 + 356) \times 64 = \sim 39K$ user/item embedding parameters, of which only 2,535 are observed; the remaining $\sim 36K$ are entirely determined by SGD initialization, optimizer trajectory, and stop conditions. There is simply not enough signal for a good local optimum. Our results in Table 5.2 are consistent with this hypothesis: deep models match or beat us only on the largest dataset (Instruments), and even on Books (the largest by interaction count) we still beat LightGCN by +113%.

(2) Inductive bias from a frozen 1B-parameter language model. The SBERT content prior provides strong inductive bias exactly where the collaborative signal is weakest: items with few co-occurrences. EASE without this prior (EASE-pure in our tables) underperforms BEST-Rec v4 by 21-47% on the small datasets but only by 2% on Books, where collaborative co-occurrence is already informative. The prior is essentially “free” collaborative information distilled from a model (Sentence-BERT, all-MiniLM-L6-v2) trained on 1B+ sentence pairs in a general-purpose contrastive setup. Crucially, this is a one-time precomputed feature: SBERT is frozen and the inference is amortized across all folds and all baselines.

(3) Determinism, reproducibility, and engineering practicality. EASE’s closed-form solution is rotation-invariant in the latent space (no random initialization), deterministic, and reproducible bit-for-bit across runs. This makes hyperparameter sensitivity easier to analyze (no SGD noise to average across), enables clean ablations (a single matrix change isolates the contribution of every component), and improves engineering practicality (the model can be retrained at every batch insert with negligible cost). Production recommender systems incurring continuous catalog churn benefit substantially from these properties — a bug or distribution shift can be diagnosed by re-running the closed-form solver and comparing to a reference, whereas a deep model’s behavior depends on the entire optimization trajectory.

6.2 The LC2C Cold-Item Algorithm

LC2C-direct (V2) is, to our knowledge, the first cold-item method that learns the per-dataset linear map from textual content to the full collaborative behavior vector of EASE. Two observations from the ablation in Section 5.5 are noteworthy.

(1) The SVD-compressed variant (V1) underperforms the direct variant (V2) on every dataset. The intuition that compressing to $k = 64$ retains signal while reducing variance is wrong: B_{warm} on Books has effective rank well above 64, and the SVD truncation discards high-frequency components of the collaborative profile. Ridge regression directly to the full n_{warm} -dimensional behavior vector preserves these components, while the μ -regularization prevents overfitting via the much smaller 384×384 normal equations.

(2) Removing the ridge mapping entirely (V3) collapses LC2C's benefit. V3 uses the nearest warm item by SBERT cosine similarity as the projector and so reduces to a fixed content-similarity-based extension of B_{warm} . Performance on Instruments and Books regresses close to content-direct. This confirms that the *learned* mapping is the source of the gain — not the SVD step, not the EASE-warm training, and not the SBERT embeddings on their own. The mapping captures dataset-specific regularities that no fixed similarity (raw cosine, lexical similarity) can express. Concretely, LC2C learns that two items with similar SBERT-titles in the Books dataset behave differently from two items with similar SBERT-titles in the Beauty dataset.

6.3 Statistical Significance and Confidence Intervals

Reporting paired-Wilcoxon p-values rather than only point estimates avoids over-claiming tiny gains. For example, on Instruments our model and Higher-Order EASE differ by 0.001 in mean NDCG@10 (Table 5.2), a difference smaller than the per-fold standard deviation. The Wilcoxon test correctly identifies this as not significant ($p = 0.55$), confirming that “adding B^2 ” is statistically equivalent to our content prior on this dataset. Similar caution is required for the cold-user protocol: the per-dataset best method (Table 5.3) varies because of legitimate dataset properties, not because of any one method's superiority.

On Books, where N is largest ($\sim 14K$ test pairs per fold), the paired Wilcoxon test detects even very small per-user differences as highly significant ($p < 1e-30$ vs. MultiVAE and LightGCN despite NDCG gaps of 0.05). Researchers should be cautious here: a Wilcoxon test with sufficiently many pairs will detect tiny effect sizes; we recommend reporting both the p-value and the absolute NDCG@10 difference for each comparison, as we do throughout.

6.4 Comparison to the Original BEST-Rec

Compared to the original BEST-Rec architecture (BERT-embedded self-attention transformer with cross-attention fusion, Section 3 of the original paper), v4 is a substantial simplification: there are no transformer layers, no learned embeddings except the four ridge-regression coefficients, and no gradient-based optimization. The empirical advantage is consistent across all four datasets and across warm, cold-user, and cold-item evaluation protocols. We attribute this gap to the parameter-to-data mismatch noted in Section 6.1: a transformer with hundreds of thousands of parameters cannot be reliably trained on 2,500 to 600,000 interactions, regardless of how well-engineered the cross-attention fusion is.

Section 7 acknowledges this design trade-off honestly: BEST-Rec v4 is not designed to scale to billions of interactions where deep recommenders likely re-emerge as the better choice due to GPU parallelism and amortized expressive capacity. Our contribution is the empirical demonstration that on small-to-medium Amazon subsets — the most common deployment scale for niche e-commerce verticals — the closed-form linear model with content prior is the right baseline.

6.5 Reproducibility and Open Science

All code, preprocessed data caches, hyperparameters, results, ablations, and figures are released as a single Jupyter notebook (BEST_Rec_v4.ipynb) accompanied by reproducible Python scripts in a uv-managed virtual environment (pyproject.toml + uv.lock pin every transitive dependency, including PyTorch CUDA 12.8). To reproduce a full evaluation, set DATASET = 'beauty' (or 'fashion', 'instruments', 'books') in the notebook's config cell and run all cells; expected runtime is 5-30 minutes depending on the dataset (dominated by the LightGCN baseline). All figures (PNG and PDF) are bundled alongside the source data JSONs for re-rendering or embedding into derivative documents.

7. Limitations and Future Work

BEST-Rec v4 has five notable limitations that suggest avenues for future research.

7.1 Item Catalog Scaling

EASE's matrix inversion is $O(n^3)$ in the item count. Our largest dataset (Books, $n = 13,164$) takes ~ 9 seconds per fold; an industrial catalog with $n \geq 10^6$ items would require 10^9 times more compute and over 1 PB of memory for the dense B matrix. The closed-form inversion is incompatible with this scale.

Several extensions could mitigate this. **Iterative Krylov methods** (conjugate gradient, GMRES) compute $X B$ without explicitly forming $B = G^{-1}$: each query requires $O(n)$ matrix-vector products with G' instead of inverting it. **Block-diagonal partitions** exploit the Amazon catalog hierarchy — items can be grouped by category and a separate EASE solved per category — trading global coverage for tractability. **Randomized low-rank approximations** (Halko-Martinsson-Tropp) reduce the inversion to a fixed-rank factorization. We have not implemented any of these and acknowledge the catalog scaling limitation as the most material gap for industrial deployment.

7.2 Cold-User Beyond Single-Item Context

Our cold-user protocol assumes the user supplies one signup signal — a search query, an interest selection, or an initial review. Pure cold-users (no signal at all) are upper-bounded by popularity, which our results confirm. Production systems with richer user metadata (demographics, device type, geographic context) could integrate that signal as additional input to a hybrid model, which we have not explored here.

A natural extension is **prompt-tuned cold-user contextualization**: an LLM produces a concise interest summary from a user's signup form, which is then SBERT-encoded and used as c_u . The few-shot context in our protocol could also be averaged across multiple items rather than a single one, smoothing the user representation when more than one signal is available.

7.3 Per-Baseline Hyperparameter Tuning

Each baseline (LightGCN, MultiVAE, iALS, Higher-Order EASE) is run at the hyperparameters reported in its original paper. A more demanding reviewer could insist on a full per-baseline grid search per dataset. Our ablation budget did not permit this; we acknowledge it explicitly. We argue that the gap of 2-3x over LightGCN/MultiVAE on most datasets is large enough that modest per-baseline tuning would not reverse the conclusion. We also note that BEST-Rec v4's own hyperparameters are tuned on 3-fold cross-validation, not on the test fold, eliminating any unfairness in the comparison.

7.4 Sequential Models

Recent recommender benchmarks include sequential models such as SASRec (Kang and McAuley, 2018) and BERT4Rec (Sun et al., 2019). These methods model the temporal sequence of user interactions and predict the next item, exploiting interaction order and timestamp signals. Amazon Reviews 2023 timestamps are coarse and incomplete — review dates are often self-reported, batch-imported, or missing — so we did not run a rigorous sequential evaluation. Future work should extend our protocol to a dataset with reliable timestamps (e.g., MovieLens-25M) and compare BEST-Rec v4 against SASRec / BERT4Rec head-to-head. We expect EASE+SBERT to remain competitive on warm leave-one-out (which does not require temporal information), but lose to sequential models on next-action prediction.

7.5 Multi-Modal Item Content

Only item titles are used as content. Item descriptions, images (CLIP embeddings), and category structure are also available in the Amazon Reviews 2023 metadata and could be aggregated into a richer S_{content} prior. We have not measured the gain. The design of LC2C is general — it learns a mapping from any d_s -dimensional content vector to the collaborative latent space — so multi-modal extensions require only the construction of an enriched content embedding, not a re-derivation of the algorithm.

An obvious next step is a multi-modal SBERT-CLIP-numeric concatenation: e_i becomes [SBERT(title); CLIP(image); $z(\text{price}, \text{avg_rating}, \text{rating_count})$]. The dimension grows to $\sim 512 + 768 + 3 = \sim 1,283$ but remains tractable for the ridge regression and the cosine similarity matrix.

8. Conclusion

BEST-Rec v4 is a closed-form linear recommender that augments the EASE algorithm of Steck (2019) with an SBERT-based content prior and introduces LC2C-direct (Section 3.4) — a novel learned ridge-regression mapping from textual content to collaborative behavior vectors — for cold items with no training interactions. Across four Amazon Reviews 2023 categories spanning three orders of magnitude in size (Beauty 2.5K interactions to Books 600K interactions), BEST-Rec v4 statistically beats six baselines including the most-cited modern deep recommenders (LightGCN, MultiVAE, iALS) on warm leave-one-out NDCG@10, with a 78-113% gain over LightGCN at the extremes of the size spectrum (Beauty and Books). On true cold-item evaluation via GroupKFold-by-item, LC2C-direct achieves 19-141% NDCG@10 improvement over the content-direct content-KNN baseline, with the largest gain on the largest dataset (Books, +141%). On cold-user few-shot evaluation, content-based scoring wins on three of four datasets, with rank-fusion of popularity and content as a dataset-agnostic robust default.

The design is deliberately simple: zero learned embeddings, zero gradient steps, four ridge-regression coefficients in the rating head, sub-second training on small datasets, and ~ 10 seconds on the largest. This simplicity is the source of the model's robustness: with no over-parameterization, BEST-Rec v4 generalizes from extremely sparse training data without the heavy regularization, dropout, or architectural innovations required by deep recommenders. The practical implication is that for small-to-mid Amazon categories, a well-engineered closed-form recommender is the appropriate baseline; the deep alternatives that proliferate in the recommender literature do not yet justify their complexity at this data scale.

The ablation studies in Section 5 systematically isolate the contribution of every component: the SBERT content prior (Section 5.6 vs. random embeddings, +33% on Beauty), the ridge regression in LC2C (Section 5.5 vs. nearest-warm V3, restored cold-item gains), the absence of SVD compression (Section 5.5 V2 vs. V1, +43% on Books), and the β weight (Section 5.4 sensitivity heatmap, robust at $\beta = 10$). Each empirical finding is paired with a theoretical explanation in Section 6.

All code, preprocessed data, results, and figures are released alongside this paper. Researchers reproducing this work need only to clone the repository, install the uv-managed Python environment, set DATASET in the notebook's config cell, and run all cells. The pipeline is fully deterministic; the same dataset run twice produces bit-identical results.

We hope this paper contributes to the growing literature on simplicity-first recommender design, in line with reproducibility studies (Dacrema et al., 2019; Rendle et al., 2020) that have repeatedly found tuned linear baselines competitive with state-of-the-art deep models. BEST-Rec v4 extends this position with an explicit cold-item algorithm (LC2C) and a rigorous four-dataset evaluation protocol. We invite the community to build on the LC2C framework: any improvement to the content-to-CF mapping (deeper neural networks, attention over warm items, multi-modal content) plugs directly into the same architecture and evaluation protocol.